

DarkPulse Low-Cost AWS EC2 Deployment Guide

Date: 2026-06-05

Project: DarkPulse 2.0 OSINT Threat Intelligence Dashboard

Final approach: one AWS EC2 instance, Docker Compose, local MongoDB container, Nginx reverse proxy, HTTPS with Certbot, and a GoDaddy domain pointed to the EC2 Elastic IP.

Domain placeholder: 'yourdomain.com'

1. Architecture

Use this low-cost architecture:

```
User Browser
-> https://yourdomain.com
-> GoDaddy DNS A record
-> AWS EC2 Elastic IP
-> Nginx on EC2 ports 80/443
-> FastAPI Docker container on localhost:8000
-> MongoDB Docker container on private Docker network
```

Services on EC2:

api container	FastAPI dashboard and backend
mongodb container	local MongoDB database
tor container	internal SOCKS proxy for collectors
collector container	scheduled background collectors
nginx host service	public reverse proxy and HTTPS entrypoint

This is the cheapest practical deployment for your current project because it avoids a paid AWS load balancer, avoids Route 53 hosted-zone cost, and uses the GoDaddy DNS panel you already have.

2. What This Approach Does and Does Not Do

This approach is good for:

- FYP/demo deployment.
- One public domain.
- One EC2 instance.
- Simple Docker Compose deployment.
- Keeping MongoDB and Tor private.
- Reusing your existing local 100k Mongo dataset.

This approach is not high availability:

- If EC2 stops, the app is offline.
- MongoDB is on the same instance.
- Scaling requires manual work later.
- Backups must be handled carefully.

For your current goal, this is acceptable and low cost.

3. Finish 100k Records Locally First

Your local MongoDB already had:

```
Stored records: 99,465
Records remaining to 100k: 535
```

Temporarily increase local collector concurrency if your laptop can handle it:

```
COLLECTOR_MAX_WORKERS=4
RUN_ON_SCHEDULER_START=false
```

Run a one-time local collection:

```
python3 orchestrator.py --once
```

Or run selected collectors:

```
python3 orchestrator.py --once --collector exploit
python3 orchestrator.py --once --collector news
python3 orchestrator.py --once --collector leaks
```

Check the stored-record count:

```
python3 -c 'from pymongo import MongoClient;
db=MongoClient("mongodb://127.0.0.1:27017")["darkpulse"];
cols=("agent_state","api_items","articles","automation_state","clean_intel","collector_source_status","credentials");
print(sum(db[c].count_documents({}) for c in cols))'
```

After reaching 100k, set the safer value again:

```
COLLECTOR_MAX_WORKERS=2
```

4. Commit Only Deployment-Safe Files

Do not commit '.env', Mongo data, screenshots, reports, logs, or runtime files.

Commit code/config only:

```
git status
git add .dockerignore .gitignore .env.example config.py docker-compose.yml
orchestrator.py ui_server.py AWS_EC2_DOMAIN_DEPLOYMENT_REPORT.md
tools/generate_simple_pdf.py
git commit -m "Prepare DarkPulse for low-cost EC2 deployment"
git push origin main
```

Keep 'DEPLOYMENT_COST_REPORT.md' local if it was only for your AWS vs Vercel decision.

5. Export Local MongoDB

Create a local Mongo dump after reaching 100k:

```
mkdir -p backups
mongodump --uri="mongodb://127.0.0.1:27017/darkpulse" --out
backups/darkpulse_100k_dump
```

Compress it:

```
tar -czf backups/darkpulse_100k_dump.tar.gz -C backups darkpulse_100k_dump
```

Keep this backup safe. Do not commit it to GitHub.

6. Prepare Optional Screenshot Cache

Your feed screenshots are not tracked in GitHub. That is good because screenshots can make the repo huge.

If your demo depends on the exact local cached screenshots, migrate them separately after EC2 is ready:

```
scp -i your-key.pem -r data/feed_screenshots ubuntu@EC2_IP:~/fyp/data/
```

If you do not migrate them, pictures can still appear when:

- records contain remote image URLs,
- records contain image/base64 data in Mongo,
- the backend generates screenshots on demand,
- the user opens records that trigger screenshot capture.

The frontend does not load 100k images at once. It loads only the visible page/cards and detail views.

7. Launch AWS EC2

Create an EC2 instance:

AMI: Ubuntu Server 22.04 LTS or 24.04 LTS
Instance type: 4 vCPU / 16 GB RAM, such as t3.xlarge or t3a.xlarge
Storage: 80-120 GB gp3 EBS
Key pair: create/download a .pem key

Security group inbound rules:

SSH	TCP 22	Your IP only
HTTP	TCP 80	0.0.0.0/0 and ::/0
HTTPS	TCP 443	0.0.0.0/0 and ::/0

Do not open these publicly:

MongoDB	TCP 27017
Tor	TCP 9050 or 19050
App	TCP 8000

Allocate an Elastic IP:

EC2 Console -> Elastic IPs -> Allocate Elastic IP -> Associate with your instance

Use this Elastic IP for GoDaddy DNS.

8. SSH Into EC2

From your local machine:

```
chmod 400 your-key.pem
ssh -i your-key.pem ubuntu@EC2_ELASTIC_IP
```

Update Ubuntu:

```
sudo apt update
sudo apt upgrade -y
```

Install base packages:

```
sudo apt install -y git curl ca-certificates nginx certbot python3-certbot-nginx
```

9. Install Docker on EC2

Install Docker:

```
curl -fsSL https://get.docker.com | sudo sh
sudo usermod -aG docker ubuntu
```

Log out:

```
exit
```

SSH back in:

```
ssh -i your-key.pem ubuntu@EC2_ELASTIC_IP
```

Verify Docker:

```
docker --version
docker compose version
```

Docker's official Ubuntu documentation is the reference for Ubuntu Docker Engine installation. The convenience script above is simpler for this demo deployment.

10. Clone the Project on EC2

```
git clone https://github.com/alynaeem/fyp.git
cd fyp
```

Create required runtime folders:

```
mkdir -p data/feed_screenshots data/credential_checker logs
api_collector/scripts/trivy_reports
```

11. Create EC2 '.env'

Create '.env' manually on EC2:

```
nano .env
```

Use this template and replace values:

```
MONGO_URI=mongodb://mongodb:27017
MONGO_DB=darkpulse

JWT_SECRET=REPLACE_WITH_LONG_RANDOM_SECRET

INITIAL_ADMIN_USERNAME=youradmin
INITIAL_ADMIN_PASSWORD=your_strong_password
INITIAL_ADMIN_EMAIL=your@email.com
INITIAL_ADMIN_NAME=Administrator

API_KEY=
CORS_ORIGINS=https://yourdomain.com,https://www.yourdomain.com

GITHUB_TOKEN=
OPENROUTER_API_KEY=
PAGESPEED_API_KEY=
GEMINI_API_KEY=

RUN_ON_SCHEDULER_START=false
COLLECTOR_MAX_WORKERS=2
CREDENTIAL_UPLOAD_MAX_BYTES=26214400

SCHEDULE_INTERVAL_HOURS=6
HEALING_MONITOR_ENABLED=false
LOG_LEVEL=INFO
```

Generate 'JWT_SECRET':

```
python3 -c "import secrets; print(secrets.token_urlsafe(48))"
```

Important:

- 'INITIAL_ADMIN_USERNAME' and 'INITIAL_ADMIN_PASSWORD' only create the first admin if MongoDB is empty.
- After your first successful admin login, remove/blank those two values and restart containers.
- New frontend users remain pending until admin approval.

12. Start Docker Compose

Build and start:

```
docker compose up --build -d
```

Check containers:

```
docker compose ps
```

Watch API logs:

```
docker compose logs -f api
```

Check health locally on EC2:

```
curl http://localhost:8000/health
```

Expected:

```
{"status":"ok","database":"mongodb connected"}
```

13. Upload Mongo Dump to EC2

From your local machine:

```
scp -i your-key.pem backups/darkpulse_100k_dump.tar.gz ubuntu@EC2_ELASTIC_IP:~/
```

On EC2:

```
mkdir -p ~/mongo_restore  
tar -xzf ~/darkpulse_100k_dump.tar.gz -C ~/mongo_restore
```

Copy dump into Mongo container:

```
docker cp ~/mongo_restore/darkpulse_100k_dump  
darkpulse-mongo:/tmp/darkpulse_100k_dump
```

Restore:

```
docker exec darkpulse-mongo mongorestore --drop --db darkpulse  
/tmp/darkpulse_100k_dump/darkpulse
```

Restart app containers:

```
docker compose restart api collector
```

Verify Mongo count:

```
docker exec darkpulse-mongo mongosh darkpulse --quiet --eval  
'db.redis_kv_store.countDocuments()'
```

14. Optional: Upload Cached Screenshots

Only do this if the demo needs your exact local cached screenshots.

From local machine:

```
scp -i your-key.pem -r data/feed_screenshots ubuntu@EC2_ELASTIC_IP:~/fyp/data/
```

On EC2, confirm:

```
ls -lh ~/fyp/data/feed_screenshots | head
```

Restart API if needed:

```
docker compose restart api
```

15. Configure Nginx

Create Nginx config:

```
sudo nano /etc/nginx/sites-available/darkpulse
```

Paste this and replace domain:

```
server {
    listen 80;
    server_name yourdomain.com www.yourdomain.com;

    client_max_body_size 30M;

    location / {
        proxy_pass http://127.0.0.1:8000;
        proxy_http_version 1.1;

        proxy_set_header Host $host;
        proxy_set_header Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;

        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
    }
}
```

Enable site:

```
sudo ln -s /etc/nginx/sites-available/darkpulse /etc/nginx/sites-enabled/darkpulse
sudo nginx -t
sudo systemctl reload nginx
```

Test EC2 HTTP directly before DNS:

```
curl http://EC2_ELASTIC_IP
```

16. Point GoDaddy Domain to EC2

Use GoDaddy DNS directly. You do not need Route 53 for this low-cost approach.

In GoDaddy:

```
GoDaddy account
-> My Products
-> Domains
-> select your domain
```

-> DNS / Manage DNS

Add or edit these DNS records:

Type: A
Name: @
Value: EC2_ELASTIC_IP
TTL: 600 seconds or default

Type: CNAME
Name: www
Value: @
TTL: 600 seconds or default

Alternative for 'www':

Type: A
Name: www
Value: EC2_ELASTIC_IP

Wait for DNS propagation. It can be quick, but allow up to a few hours.

Test from your local machine:

```
dig yourdomain.com
dig www.yourdomain.com
curl http://yourdomain.com
```

The IP returned by 'dig' should be your EC2 Elastic IP.

GoDaddy's official DNS/nameserver help explains that DNS is managed through the GoDaddy account when using GoDaddy nameservers. Keep GoDaddy nameservers for this low-cost approach and edit DNS records there.

17. Enable HTTPS With Certbot

Run this only after GoDaddy DNS points to EC2.

On EC2:

```
sudo certbot --nginx -d yourdomain.com -d www.yourdomain.com
```

Choose the redirect option when prompted so HTTP redirects to HTTPS.

Test renewal:

```
sudo certbot renew --dry-run
```

Open:

```
https://yourdomain.com
```

Certbot's Nginx plugin edits your Nginx config and installs the certificate.

18. First Login

Open:

```
https://yourdomain.com
```

Login using the initial admin credentials from '.env':

```
INITIAL_ADMIN_USERNAME  
INITIAL_ADMIN_PASSWORD
```

After successful login, edit '.env':

```
nano .env
```

Blank these:

```
INITIAL_ADMIN_USERNAME=  
INITIAL_ADMIN_PASSWORD=
```

Restart:

```
docker compose restart api collector
```

The admin account remains in MongoDB.

19. Verify Deployment

Check app:

```
curl https://yourdomain.com/health
```

Check containers:

```
docker compose ps
```

Check logs:

```
docker compose logs -f api  
docker compose logs -f collector
```

Check database:

```
docker exec darkpulse-mongo mongosh darkpulse --quiet --eval  
'db.redis_kv_store.countDocuments()'
```

In the frontend:

- Login works.
- Dashboard Total shows 100k+ after restore.
- Live Feed loads.

- Admin user page works.
- Screenshot/image behavior is acceptable.
- New registrations stay pending until admin approval.

20. Backups

Do not rely only on EC2 existing. Create backups.

Mongo backup on EC2:

```
docker exec darkpulse-mongo mongodump --db darkpulse --out /tmp/darkpulse_backup
docker cp darkpulse-mongo:/tmp/darkpulse_backup ~/darkpulse_backup
tar -czf ~/darkpulse_backup.tar.gz -C ~ darkpulse_backup
```

Download backup locally:

```
scp -i your-key.pem ubuntu@EC2_ELASTIC_IP:~/darkpulse_backup.tar.gz backups/
```

Optional EC2-level backup:

```
EC2 Console -> Volumes -> select EBS volume -> Create snapshot
```

Create EBS snapshots after major successful data imports.

21. Updating Code Later

On EC2:

```
cd ~/fyp
git pull origin main
docker compose up --build -d
docker compose logs -f api
```

Do not overwrite '.env' during updates.

22. Troubleshooting

Docker containers:

```
docker compose ps
docker compose logs -f api
```

Nginx:

```
sudo nginx -t
sudo systemctl status nginx
sudo systemctl reload nginx
```

HTTPS:

```
sudo certbot certificates
sudo certbot renew --dry-run
```

DNS:

```
dig yourdomain.com
dig www.yourdomain.com
```

Disk:

```
df -h
docker system df
```

If disk gets full:

```
docker system prune
```

Do not run destructive cleanup on Mongo volumes.

23. Final Checklist

- Local Mongo reaches 100k+.
- Local Mongo dump created.
- Code pushed to GitHub.
- EC2 instance created.
- Elastic IP associated.
- Security group exposes only SSH from your IP plus HTTP/HTTPS publicly.
- Docker installed.
- Repo cloned.
- '.env' created on EC2.
- Docker Compose is running.
- Mongo dump restored.
- Optional screenshots migrated only if needed.
- Nginx proxies to '127.0.0.1:8000'.
- GoDaddy 'A @' record points to EC2 Elastic IP.
- GoDaddy 'www' record points to root or EC2 Elastic IP.
- Certbot HTTPS is active.
- Admin login works.
- Bootstrap admin values removed from '.env'.
- Dashboard Total shows 100k+.
- Mongo backup created.

References

- GoDaddy DNS/nameserver management:
<https://www.godaddy.com/help/change-nameservers-for-my-domains-664>
- Docker Engine on Ubuntu: <https://docs.docker.com/installation/ubuntulinux/>
- Certbot with Nginx: <https://certbot.eff.org/>